Lavoro parallelo sul fisso

Ragionamento, pseudocodice e strumenti Unreal

Ramo `claude/project-thread-v1plhd`

25 ottobre 2026

Indice

1 Cosa c'è in questo documento **1**

2 Muri: step up e step down separati **1**
2.1 Ragionamento . . . 1
2.2 Pseudocodice . . . 2
2.3 Strumenti Unreal . . . 2
2.4 Test . . . 2
2.5 Costo delle query . . . 2

3 Unione del ramo M4 **2**

4 Editor dei combattimenti (M5): i passi che aspettavano M4 **2**
4.1 E3: fasi derivate del colpo (`f100773`) . . . 3
4.2 E11: regole di validazione M4, "Validate all" e pannello (`5a31adc`) . . . 3
4.3 E5b: righe Hitbox e HurtboxModifier nella timeline (`281fa38`) . . . 4
4.4 E12: riga dei dati del colpo e anteprima del knockback (`58af658`) . . . 4
4.5 E8: bake dei socket e controllo di freschezza (`931dc89`) . . . 5
4.6 E9: editor dei set di hurtbox, generatore e specchio (`4b325c8`) . . . 5
4.7 E7: gizmo e selezione nel viewport (`25c1878`) . . . 6

5 Muri in movimento: correzioni di rete che convergono (`1ba37ee`) **7**
5.1 Causa . . . 7
5.2 Correzione . . . 7
5.3 Numeri . . . 7
5.4 Aperto . . . 7

6 Cosa controllare a mano stasera **7**

1 Cosa c'è in questo documento

Il lavoro fatto sul fisso mentre il portatile porta avanti M4: lo step up/step down dei muri, l'unione del ramo M4 e i passi dell'editor dei combattimenti (M5) che aspettavano i dati di M4. Per ogni parte: perché è fatta così, lo pseudocodice della logica principale, le classi e gli strumenti di Unreal usati, i test e cosa resta da controllare a mano nell'editor (sezione 6).

2 Muri: step up e step down separati

2.1 Ragionamento

Il look-ahead del wall slide v2 guarda la parete davanti al personaggio e classifica ciò che trova (`EGSTN_WallEdgeKind`): *Bump* (continuo, si scavalca), *Slope*, *Concave* (angolo interno), *Convex* (spigolo esterno), *Blocked*. Prima c'era un solo limite, `MaxStepHeight`, usato sia per ciò che sporge dal muro sia per ciò che rientra. Come nella modalità di camminata, ora i limiti sono due: **step up** (`MaxStepHeight`, mostrato come "Max Step Up Height") e **step down** (`MaxStepDownDepth`), entrambi 10 cm di default, quindi il comportamento non cambia finché un designer non li modifica. Il nome della proprietà esistente è rimasto lo stesso, così i valori già salvati nei Blueprint si caricano senza redirect.

Nota sulla rientranza da 15 cm: il personaggio ci entra (l'adesione lo tira verso la parete) e ne esce risalendo di 15 cm; quella risalita è uno *step up*. Per attraversarla senza rallentare va alzato lo step up a 15 cm o più.

2.2 Pseudocodice

```
ClassifyAhead(Offset, Angolo, ...):
  // Offset < 0: la superficie sporge verso il personaggio; > 0: rientra
  Limite = (Offset <= 0) ? MaxStepHeight : MaxStepDownDepth
  se |Offset| <= Limite:
      se l'angolo non èdolce -> Bump (si scavalca, mai angolo)
      se |Offset| quasi zero -> None
      se l'offset èquello di un pendio -> Slope, altrimenti Bump
  altrimenti se angolo dolce -> Slope
  altrimenti se angolo oltre il limite -> Blocked
  altrimenti -> Offset > 0 ? Convex : Concave
```

2.3 Strumenti Unreal

`USTRUCT FGSTN_WallEdgeSettings` con `UPROPERTY(EditAnywhere, meta=(DisplayName, ClampMin, ForceUnits="cm"))`; la funzione pura in `UGSTN_WallEdgeUtils` (`UBlueprintFunctionLibrary`); test con `IMPLEMENT_SIMPLE_AUTOMATION_TEST`.

2.4 Test

`GSTN.Wall.Edge.StepUpAndDownAreSeparate` (classificazione con limiti diversi) e `GSTN.Wall.V2.Lane.RecessWithinTheStepUpIsNotACorner` (la rientranza da 15 cm, con i limiti a 20 cm, si attraversa a velocità piena). 74 test dei muri verdi. Commit `8be7b4d`.

2.5 Costo delle query

Il campionamento che distingue una curva da uno spigolo porta il v2 a 5–5,7 query per tick (massimo 9) sulle corsie con curve e angoli, contro le 3,5 della parete piatta. Accettato per ora; se il wall slide comparirà in un profilo, i primi interventi sono campionare la catena un tick sì e uno no e tenerla in cache finché lo stesso angolo resta nel look-ahead (`wall-slide-v2.md` §6.13).

3 Unione del ramo M4

Il ramo del portatile (`feature/gas-npp-5.8`, M4 fino a H13b) è stato unito nel ramo del fisso (`5d0b7da`). Un solo conflitto, l'elenco dei tag di combattimento in `GSTN_CombatTags.cpp`: tenuti i tag di entrambi i lati. `Plugins/GSTNSurface` è identico sui due rami. 806 test verdi dopo l'unione.

4 Editor dei combattimenti (M5): i passi che aspettavano M4

Quattro passi del piano `m5-editor-plan.md`, uno per commit, tutti sul plugin `GSTNCombatEditor` tranne E3 (che vive nel Core di combattimento). Test: da 806 a 846, tutti verdi. Le deviazioni dal piano sono in `docs/notes/m5-doc-deltas.md`.

4.1 E3: fasi derivate del colpo (`f100773`)

Ragionamento. Startup, attivo e recovery di una mossa non si scrivono a mano: si ricavano dalle tracce hitbox della timeline, così il dato mostrato non può divergere da ciò che il gioco esegue. `FGSTN_TimelinePhases` sta nel Core (`GSTNCombatCore`, `Timeline/GSTN_TimelinePhases.h`) perché è logica pura, utile anche fuori dall'editor; la barra delle fasi dell'editor (`FGSTN_PhaseBarData`) resta solo un modello di disegno e legge questi numeri senza ricalcolarli. Una traccia hitbox senza specifiche conta come attiva (è un guscio creato da un autore: lo segnala il validatore); una traccia le cui varianti ignorano tutte e quattro le relazioni non conta.

```
finestre = [(max(inizio,0), fine) per ogni traccia Hitbox con fine > inizio e
    ContaComeAttiva(traccia)]
ordina per inizio; unisci se prossima.inizio <= ultima.fine
Attivo = prima.inizio .. ultima.fine ; Startup = prima.inizio ; PrimoColpo (1-based) =
    prima.inizio + 1
Recovery = max(FAF - FineAttivo, 0) ; nessuna finestra -> bHasActive = false
```

Unreal. `USTRUCT` nel Core, `FText` per il riassunto (`FormatSummary(TickRate)`), array riservato una volta sola (nessuna allocazione per mosse senza hitbox). **Test**: `GSTNCombat.Core.TimelinePhases.*` (10).

Da notare. A 62,5 Hz 30 frame sono 480 ms; l'esempio della specifica diceva 484 (calcolato a 62 Hz).

4.2 E11: regole di validazione M4, "Validate all" e pannello (`5a31adc`)

Ragionamento. Stesso schema di E10: funzioni pure che riempiono un report, chiamate sia dalla Data Validation di Unreal sia dal pannello, così pannello e salvataggio dicono sempre la stessa cosa (`UGSTN_CombatValidatorBase::BuildReportForAsset`). Dove il Core avvisa già (socket non baked, lunghezza del bake, Armor senza soglia) il suo messaggio viene classificato e non duplicato.

```
HitboxKeys: frame della chiave < 0 o >= lunghezza finestra -> errore
  stesso HitGroup, finestre sovrapposte, stessa Priority, Data diversi, forma diversa ->
      avviso
HitData: danno/knockback/crescita/moltiplicatori/stagger/hitstop negativi -> errore
  FixedKnockback >= 0 insieme a crescita -> avviso; HitType o Cue non registrati ->
      errore
  le varianti che sovrascrivono campi sono controllate allo stesso modo
Varianti: chiave fuori da Combat.Relationship, ShapeScale <= 0 senza ignore -> errore
HurtState: Armor con soglia <= 0 e non infinita -> errore
BakedSocketMissing / BakedLength -> errore ; BakedStale -> info
Hurtbox set: vuoto, nomi doppi, estensioni, tag fuori da Combat.Hurtbox -> errore
```

Pannello. `GSTNValidateAll` raccoglie gli asset (salta `/Engine` e `/Temp`) e trova i Blueprint di abilità tramite `GetDerivedClassNames` e il tag `GeneratedClass` dell'Asset Registry, *senza caricarli*. Il modello `FGSTN_ValidationPanelModel` non ha widget (testabile da solo); `SGSTN_ValidationPanel` è un `SCompoundWidget` con `STreeView`. Doppio clic: apre l'asset e porta la timeline sul frame e sulla chiave del problema. La scheda Frame Data rivalida 250 ms dopo

l'ultima modifica (`FTSTicker`) e le corsie mostrano una linea rossa o gialla. Registrazione: scheda nomade nel gruppo "GSTN Combat", comando `gstn.Combat.ValidateAll`, pagina del Message Log.

Test. `GSTNCombat.Editor.Validate.*` (14) e le istantanee `GSTN.Visual.Validation.Panel`, `.LaneMarkers`.

Rimandato. "Bake all" e la parte hash di `BakedStale` (servono il baker di E8); controlli su scheletro e ossa (servono la mesh dell'hurtbox set, E9); la fascia "clank" della Priority (è di M6).

4.3 E5b: righe Hitbox e HurtboxModifier nella timeline (`281fa38`)

Ragionamento. Una riga per traccia con una sotto-corsia per ogni hitbox, invece di una riga per hitbox: la matematica di righe, selezione e trascinamento del modello E2/E4 assume righe da 24 px, e cambiarla avrebbe toccato tutto il modello. Le chiavi restano relative all'inizio della traccia (`FGSTN_HitboxKey::Frame`), quindi spostare la finestra sposta le chiavi.

```
AddKey su un percorso: chiave = UGSTN_HitShapeUtils::EvaluateKeys(chiavi della corsia,
    frame relativo)
   // la stessa funzione del runtime: aggiungere una chiave non cambia il percorso
AddKey senza corsia: corsia della prima chiave selezionata sulla traccia, altrimenti
    corsia 0
Duplica / incolla: le copie restano nella corsia d'origine, in ordine di corsia
    crescente
Nuova traccia Hitbox: una sfera e una chiave; parte vuota di proposito, quindi nasce
    come errore
```

Unreal. Interfaccia dell'adapter estesa con `GetLaneCount`, `GetKeyLane`, `AppendKeyInLane` (con default), colore per stato tramite il nuovo virtuale `GetTrackColor`. **Test**: 8 sul modello e sulla registry, istantanea `GSTN.Visual.Timeline.HitboxRows`. **Limiti**: con cinque o più hitbox la riga è affollata (il pannello Details ha la lista completa); una chiave trascinata può uscire dalla finestra (lo segnala `HitboxKeys`).

4.4 E12: riga dei dati del colpo e anteprima del knockback (`58af658`)

Ragionamento. `FGSTN_HitDataCustomization` (`IPropertyTypeCustomization` per `FGSTN_HitData`) raggruppa i campi (danno, knockback, hitstun, hitstop, stagger, colpo, impatto, estensioni, anteprima) con un riassunto su una riga e una frase che spiega Special Angle e Yaw Source. Le estensioni usano l'editor di array standard, così la customization di un altro plugin compare dentro. Il quadrante dell'angolo (`SGSTN_AngleDial`) è un solo passo di undo: transazione aperta alla pressione, scritture interattive, chiusura al rilascio; Esc ripristina.

Anteprima. Non reimplementa la formula: costruisce lo stesso input del passo runtime `UGSTN_HitStep_ComputeKnockback` e chiama il modello delle regole di default, quindi non può dare numeri diversi dal gioco.

```
input = {colpo, su=+Z, peso, percentuale, salute 1, soglia stagger = regole.
    StaggerThreshold}
se Sakurai: K = modello(input con direzione nulla).Knockback
input.Direzione = LaunchDirection(yaw, angolo, angolo speciale, su, avanti=+X, a terra,
    K, regole.Sakurai)
risultato = modello.Compute(input) // knockback, tier, stagger, hitstun, hitstop, angolo
     effettivo
```

Unreal. `IPropertyTypeCustomization`, `SGameplayTagCombo` filtrato su `Combat.HitType` (nuova dipendenza dal plugin `GameplayTagsEditor`), transazioni dell'editor e `IPropertyHandle::SetValue` per il quadrante. **Test**: `GSTNCombat.Editor.HitData.*` (4, compreso il caso a mano $K = (400 + 10 \cdot 6) \cdot 100/50 = 920$) e l'istantanea `GSTN.Visual.HitData.Row`. **Limiti**: la vittima di prova (peso, percentuale) vive nella riga ed è solo di sessione; l'anteprima ignora scale delle hurtbox, DI, tumble e yaw diversi dall'orientamento dell'attaccante; usa solo le regole di default.

4.5 E8: bake dei socket e controllo di freschezza (`931dc89`)

Ragionamento. Le hitbox in spazio `BakedSocket` seguono un socket (la mano, la lama) senza leggere la mesh durante la simulazione: le posizioni per frame si calcolano nell'editor e si salvano nella timeline (`BakedSockets`). Il baker usa la stessa posa dell'anteprima (`FGSTN_PreviewPose`), quindi ciò che si vede è ciò che si salva; ogni frame si valuta da solo (tempo del montage impostato, mai fatto avanzare), quindi due bake sono identici bit per bit. Un bake rifiutato (mesh mancante, socket assente, montage non caricabile) non scrive nulla e tiene il precedente. Il registro del bake (`FGSTN_BakeInfo`) è solo editor (`WITH_EDITORONLY_DATA`), non arriva nel gioco.

```
usi = hitbox con Space == BakedSocket e un socket ; socket = distinti, in ordine
per f in 0..TotalFrames-1:
    tempo montage = InizioSezione + f / TickRate * PlayRate // inizio del frame f,
        limitato alla durata
    Posa.Valuta(tempo) ; per ogni socket: T = socket in spazio componente * MeshToRoot ;
        scala 1 ; aggiungi
Info = {hash degli input, tick rate, mesh, data}
hash = CRC32 di: frame, tick rate, play rate, sezione, montage e suoi segmenti (GUID dei
     dati animazione),
       mesh + GUID dello scheletro, socket ordinati con osso e trasformazione
stato: nessun uso -> NonServe ; frame mancanti -> MaiBaked ; nessun registro ->
    BakedAltrove (avviso)
       frame != TotalFrames, tick rate, mesh o hash diversi -> Vecchio (errore) ;
           altrimenti Fresco
```

Verificato sul codice del fork che `PlaySyncedMontage` avanza il montage di 16 ms per tick a 62,5 Hz con lo stesso play rate: la formula coincide. Restano due differenze accettate: il blend-in del montage in gioco (il bake è a peso pieno) e un possibile frame di sfasamento all'attivazione, da confermare con BakeCheck (E16).

Unreal. `UDebugSkelMeshComponent`, sezioni e segmenti di `UAnimMontage`, `IAnimationDataModel::GenerateGuid` (`GetRawDataGuid` è deprecato in 5.8), `USkeletalMeshSocket`, `FCrc::StrCrc32`, `FScopedSlowTask`, comando `gstn.Combat.BakeAll`. Nella toolkit: pulsante "Bake sockets" con pallino giallo, Ctrl+B, un solo undo, bake al salvataggio. **Test**: 13 (`GSTNCombat.Editor.Bake.*`, `GSTNCombat.Core.TimelineData.*`).

Non fatto. Voce di menu di Bake all (c'è il comando da console), bake dei socket delle mesh allegate (solo mesh principale), timeline inline nei Blueprint dentro Bake all (si fanno dal loro editor). Lo scenario di rete `scenario_hit_basic` va eseguito prima di unire M5 (serve il PIE).

4.6 E9: editor dei set di hurtbox, generatore e specchio (`4b325c8`)

Ragionamento. Un set di hurtbox si apre con doppio clic in un editor con viewport, lista, dettagli e problemi. Il generatore legge il Physics Asset della mesh di anteprima e tratta ogni scheletro allo stesso modo (un ragno come una persona: nel codice non c'è nessun nome di osso). Rigenerare non deve cancellare il lavoro a mano: di default aggiunge solo le hurtbox mancanti, "Update shapes" aggiorna forma e osso tenendo tag e priorità, "Replace all" chiede conferma;

l'ordine dell'array non cambia (un colpo porta l'indice). Lo specchio segue la tua risposta 2A: solo se lo scheletro ha coppie sinistra/destra riconoscibili (Auto), disattivabile o con coppie proprie per scheletro nelle impostazioni; altrimenti il pulsante è disattivo e il tooltip dice perché. La mesh di anteprima del set è versionata, perché il validatore controlla le ossa su di essa.

```
generatore: per ogni corpo del Physics Asset, per ogni primitiva:
   sfera r -> Sfera(r) ; box (X,Y,Z) -> Box(X/2,Y/2,Z/2) ; capsula (R,L) -> Capsula(R, R
       + L/2)
   convessa -> capsula sul lato più lungo del suo bounding box + avviso ; affusolata ->
       raggio maggiore + avviso
   Locale = PrimitivaLocale * OssoInSpazioComponente * MeshToRoot ; estensione * margine
   nome = osso (osso_N se più di una) ; tag = prima regola di zona il cui frammento ènel
       nome dell'osso
specchio, nomi: cerca il frammento della coppia ("_l" / "_r", "Left" / "Right"...) a
    confine di parola
   ("_l" accoppia arm_l ma non arm_lower), sostituisci mantenendo le maiuscole
specchio, forma: asse laterale = Y se l'avanti è+/-X, altrimenti X ; posizione: nega la
    componente laterale
   rotazione (x,y,z,w) -> (-x,y,-z,w) ; se la controparte esiste già: sovrascrive dopo
       conferma
```

Unreal. `UPhysicsAsset`, `USkeletalBodySetup` (`FKSphereElem`, `FKBoxElem`, `FKSphylElem`, `FKConvexElem`, `FKTaperedCapsuleElem`), `FReferenceSkeleton`, `FAssetEditorToolkit`, `UAssetDefinitionDefault`, `IStructureDetailsView`, `SListView`, `FEditorUndoClient`. Il validatore aggiunge `Hurtbox.BoneMissing` e `Hurtbox.Outside`. **Test**: 18 (`GSTNCombat.Editor.Hurtbox.*`, scheletri e Physics Asset costruiti in memoria) e l'istantanea `GSTN.Visual.Hurtbox.Panel`. Totale suite: 877 verdi.

Non fatto. I gizmo per spostare le forme nel viewport sono di E7 (per ora le forme si modificano a numeri nel Details); nessun selettore di Physics Asset (si usa quello della mesh); il pannello di validazione apre il set ma non seleziona la hurtbox.

4.7 E7: gizmo e selezione nel viewport (`25c1878`)

Ragionamento. Le forme si spostano, ruotano e scalano con il gizmo standard di Unreal, sia nella timeline (hitbox) sia nell'editor dei set di hurtbox (che così non è più solo numerico). Il gizmo di Unreal non si disegna senza una modalità di editing attiva: come il Physics Asset editor, il viewport registra una piccola `FEdMode` propria. Il gizmo sta sul percorso della hitbox al frame del playhead (non su una chiave) e compare solo dentro la finestra della traccia. Il trascinamento accumula i delta dal momento della pressione e applica il totale allo stato iniziale: niente deriva da arrotondamenti, ed Esc ripristina esattamente. La scala scrive l'estensione della forma (condivisa da tutte le chiavi), mai una scala di trasformazione. Aggiunto anche il disegno delle hitbox attive nel viewport (colorate per gruppo), che E6 non faceva.

```
Click: proxy sotto il cursore -> seleziona ; altrimenti raggio contro le forme attive
      -> la più vicina (a pari distanza la più piccola) ; seleziona la chiave al
          playhead o la più vicina
Inizio drag: chiave sul frame -> modificala ; nessuna e auto-key -> crea la chiave (
    stesso passo di undo)
           nessuna e auto-key spento -> modifica la più vicina ; Shift -> sposta tutte
               le chiavi della corsia
Durante il drag, S = spazio della hitbox (Root, socket baked o vivo, origine della mossa
    ):
   Offset = Inizio + S.InverseTransformVector(deltaMondo)
   Rot = S.rot^-1 * rotMondo * S.rot * Inizio.Rot
   Estensione = ScalaEstensione(formaIniziale, unità) // 1 + unità * 0.01 per asse
```

```
Mirror: chiave in spazio root, x -> -x, quat (x,y,z,w) -> (x,-y,-z,w)
```

Unreal. `FEditorViewportClient` (`InputWidgetDelta`, `ProcessClick`, `AbortTracking`, `GetWidgetLocation`), `UE::Widget::EWidgetMode`, `HHitProxy/DECLARE_HIT_PROXY`, `HWidgetAxis`, `FEdMode` con `FEditorModeRegistry`, `FScopedTransaction`. **Test**: 7 (conversioni di spazio, scala sull'estensione, auto-key in un solo undo, selezione, mirror, drag delle hurtbox). Totale suite: 884 verdi. **Da tarare**: il fattore 0,01 della scala. **Non fatto**: voci Mirror/Offset nel menu delle chiavi (ci sono pulsante e Shift), snap della scala, overlay delle hurtbox del kit nella timeline.

5 Muri in movimento: correzioni di rete che convergono (`1ba37ee`)

5.1 Causa

Sui muri che si muovono il personaggio "cavalca" la parete: ogni tick viene portato alla posa attuale della base moltiplicata per la sua posizione locale sulla parete (`FGSTN_WallBaseInfo::LocalPosition`). Quella posizione locale era scritta a fine tick in un *rollback blackboard* di Mover. In una correzione, Network Prediction ripristina lo stato sincronizzato del server (la capsula torna dove la vuole il server), ma il blackboard torna al valore del frame prima *come lo aveva predetto il client*: il primo tick risimulato riportava la capsula nel vecchio punto del client, annullando la correzione, e lo sfasamento (circa 20 cm lungo la corsa del muro) restava per sempre. Non era l'aggancio in sé: qualunque correzione durante la cavalcata lo causava; un aggancio vicino al teletrasporto del test lo rendeva certo.

5.2 Correzione

La posizione locale non si conserva più: si ricava ogni tick dalla posizione sincronizzata e dalla posa della base al frame prima. Attivo sulle modalità v2 (`bRideFromSyncedLocation`, vero nel costruttore di `UGSTN_WallTraverseModeBase`); spento di default sulle modalità legacy, che quindi restano come prima.

```
prec = Blackboard[BaseInfo] // componente base e sua posa a fine frame precedente
inizio = posizione dello stato sincronizzato all'inizio del tick
locale = prec.Rotazione^-1 * (inizio - prec.Posizione)
target = BaseOra.Posizione + BaseOra.Rotazione * locale
sposta la capsula su target (non swept, come prima); ruota normale, velocità e capsula
    del delta di rotazione della base
```

È sicuro per il rollback perché dipende solo dallo stato sincronizzato e dalla posa della base, che è uguale su ogni macchina per basi posate dal tempo di simulazione (come `AGSTN_WallTestMover`). Senza correzioni il risultato è identico a prima.

5.3 Numeri

Corsia, modalità	Prima (aggancio presto)	Dopo (presto / partenza normale)
`h_moving150` v2	36 riconciliazioni	3 / 1 (0 dopo 0,6 s)
`h_moving300` v2	85 riconciliazioni	5 / 1 (0 dopo 0,6 s)
`h_moving150` legacy	123	124 (non toccata)

Test: `GSTN.Wall.V2Net.CorrectionOnMovingWallConverges` (errore predetto 20,22 cm, dopo la correzione 0,000 cm; il controllo con la cavalcata vecchia resta a 20,22). Suite: 885 verdi. Nuova opzione del harness: `wall_ab.py -start-gap N`. Classi: `URollbackBlackboard`, `UMoverComponent::OnSimulationRollback`, `FMoverDefaultSyncState`.

5.4 Aperto

Le modalità legacy hanno ancora il problema (opzione spenta: decisione tua). Basi la cui posa differisce tra macchine nello stesso frame (corpi fisici simulati) richiederebbero la base nello stato sincronizzato, come fa Mover: cambio di design non fatto.

6 Cosa controllare a mano stasera

1. **Muri**: in `SandboxCharacter_Mover`, modalità WallSlidingV2, sezione *Edges*: ci sono "Max Step Up Height" e "Max Step Down Depth". Sulla mappa `L_WallSlideAB`, corsia delle rientranze: con lo step up a 15–20 cm la rientranza da 15 cm non deve più rallentare.

2. **Timeline, righe hitbox**: apri un asset timeline, aggiungi una traccia Hitbox con doppio clic, trascina le chiavi tra le sotto-corsie, premi K con una chiave selezionata, Ctrl+trascina per copiare, poi annulla e ripeti.

3. **Dati del colpo**: seleziona un hitbox; nel Details trascina il quadrante dell'angolo (un solo undo, Esc ripristina), cambia il tag HitType, aggiungi un'estensione, leggi la riga di anteprima del knockback.

4. **Validazione**: crea un errore apposta (danno negativo o chiave oltre la finestra); entro circa 250 ms reagiscono la scheda Frame Data e la linea sulla corsia. Premi F7 e controlla il Message Log.

5. **Validate all**: Window, GSTN Combat, Validation, "Validate all", poi doppio clic su un problema di un asset timeline e di una timeline inline in un Blueprint di abilità: playhead e selezione devono finire sul frame giusto.

6. **Bake dei socket**: apri una timeline con una hitbox `BakedSocket` e una mesh di anteprima. Il pulsante "Bake sockets" ha il pallino giallo se mancano i frame; clic o Ctrl+B e il pallino sparisce; cambia sezione o play rate e torna; Edit, Undo ripristina il bake vecchio; confronta la hitbox con la mano nel viewport. In console: `gstn.Combat.BakeAll`.

7. **Hurtbox set**: doppio clic su un Hurtbox Set, scegli la mesh di un personaggio con Physics Asset, Generate, Add missing: le forme stanno sul corpo. Prova Update shapes e Replace all (conferma). Seleziona una hurtbox sinistra e Mirror: compare dall'altro lato. Su uno scheletro senza sinistra/destra (nemico, torretta) Mirror è disattivo con il motivo. Undo dopo ogni azione.

8. **Gizmo (E7)**: timeline con traccia Hitbox, kit e skin; playhead dentro la finestra, spunta "Hitboxes": si vedono le forme. Clic sulle linee o dentro il volume: si evidenzia la chiave e compare il gizmo. W/E/R e trascina ogni maniglia (la scala è comoda?). Ctrl+~ per Local/World (la scala resta locale). Su un frame senza chiave con auto-key: una chiave, un solo Ctrl+Z la toglie. Esc a metà drag ripristina tutto. Shift all'inizio sposta tutte le chiavi. Playhead fuori dalla finestra: il gizmo sparisce. Nell'editor delle hurtbox: clic, drag, undo ed Esc.

9. **Muri in movimento**: su `L_WallSlideAB`, corsie h, aggrappati subito dopo un salto o un dash sul muro che si muove: niente scatti o tremolii lungo la corsa. Con `np.PrintReconciles 1` nessuna discrepanza ripetuta mentre scivoli.

10. **Rete**: prima di unire M5, esegui lo scenario `scenario_hit_basic` del harness (il bake non è stato provato in PIE).